docs_chain

Revision: 1 **Last modified:** 2026-05-29T08:59:39Z **Status:** Phase 1 core engine IMPLEMENTED + tested (`internal/hash` + `internal/graph` — `go test ./...` passes). Phases 2–7 PLANNED per the plan. This README and the docs under `docs/` mark each capability IMPLEMENTED vs PLANNED and do not claim working behaviour that is not yet built (§11.4.6). **Authority:** Operator mandate 2026-05-29 (docs_chain initiative)

---

docs_chain is a universal, Go-implemented, **bidirectional document-and-database dependency-propagation engine**. When any member of a registered chain changes — a Markdown source, an HTML/PDF export, or a SQLite database — docs_chain detects the change by **content hash** (not mtime) and propagates it through every connected member in every declared direction, regenerating and re-exporting atomically, so no tracked artefact can fall out of sync.

It is the mechanical successor to the project's ad-hoc documentation-sync scripts. It ships as its own `vasic-digital` submodule and is consumed as a core part of the HelixConstitution submodule so any project adopts it out of the box and registers its own chains via per-context YAML.

Model in one line

Salsa-style content-hashed incremental recomputation over a DAG, with Kahn topological ordering, early-cutoff, declared-authority bidirectional `sync` edges, and atomic-rename + SQLite-transaction commit.

Implementation status

Phase	Scope	Status
0	Research + design + this documentation	Done (design artefacts)
1	Core DAG + content-hash engine	**IMPLEMENTED + tested** (`internal/hash`, `internal/graph`)

Phase	Scope	Status
2	Node adapters + transforms	PLANNED
3	Propagation orchestrator + atomicity (filesystem/ SQLite)	PLANNED
4	Config-driven multi-context + CLI/daemon	PLANNED
5	Comprehensive test suite (beyond the Phase 1 unit tests)	PLANNED
6	Constitution-submodule integration + repo creation	PLANNED — OPERATOR-GATED
7	ATMOSphere wiring + retire ad-hoc scripts	PLANNED

What is IMPLEMENTED today (`go test ./...` passes)

- **`internal/hash`** — `Hasher` interface + `ByteContentHasher` (LF normalization, trailing-whitespace strip, single trailing newline) so semantically-equivalent inputs collide by design; plus the sorted member-list fingerprint for roster/corpus sidecars (§11.4.86). Change detection is by **content hash, never mtime**.
- **`internal/graph`** — the DAG (`Node`/`Edge`/`Graph`), `Validate` (cycle detection → `CycleError`), `TopoOrder` (deterministic Kahn ordering), and the recompute engine: `Recompute` (early-cutoff — unchanged inputs skip transform), `ResolveSync` (declared-authority bidirectional `sync` edges with `ConflictError`), and `CommitHashes`. An in-memory `Store` (`MemStore`) backs the unit tests.

What is PLANNED (NOT yet functional — do not assume working behaviour)

The `cmd/` CLI/daemon, on-disk node adapters (Markdown→HTML/ PDF/DOCX transforms, SQLite store), the atomic-rename + SQLite-transaction commit layer, per-context YAML config loading, and the constitution-submodule integration are owned by Phases 2–7 and are NOT implemented in this repo yet. The `docs/` pages describe their DESIGNED contract.

Documentation

Document	Purpose
docs/ARCHITECTURE.md	In-depth system architecture — DAG model, content-hash detection, early-cutoff, Kahn propagation, bidirectional sync authority + conflicts, atomicity/ crash-safety, SQLite integration. Mermaid + ASCII diagrams.
docs/USER_GUIDE.md	End-to-end adoption guide — prerequisites, build, init `.docs_chain/`, define a context, run `sync`, the watch daemon, conflict resolution, CI integration.
docs/CONFIG_SCHEMA.md	Formal per-context YAML reference — every field, type, allowed values, defaults, validation rules; annotated `derive-from` + `sync` examples.
docs/USE_CASE_CATALOGUE.md	Living registry of 8 ready-to-use chain recipes (issues, fixed, status, roster/corpus, changelog, README doc-link, universal markdown-export, CONTINUATION).
docs/ CONSTITUTION_INTEGRATION.md	How the constitution submodule makes docs_chain available to every consuming project — inheritance model, config discovery, the §11.4.x anchors it satisfies, what a project gets for free vs must register.

Quick start (Phase 1 core engine)

```
git clone git@github.com:vasic-digital/docs_chain.git
cd docs_chain
go test ./...        # internal/hash + internal/graph — all green
```

The library API today is consumed programmatically (the `cmd/` CLI is Phase 4). Minimal use of the implemented core:

```
import (
    "digital.vasic.docs_chain/internal/graph"
    "digital.vasic.docs_chain/internal/hash"
)
```

```
g := graph.New()
// AddNode for each member, AddEdge for derive-from / sync
        relationships,
// then g.Validate() (cycle check) and g.TopoOrder() (Kahn
        ordering).
res, err := g.Recompute(store, hash.NewByteContentHasher(),
        transforms)
// res reports which targets changed (early-cutoff skips unchanged
        inputs);
// g.CommitHashes(res) persists the new content hashes.
```

See docs/USER_GUIDE.md for the full DESIGNED workflow and docs/CONFIG_SCHEMA.md for the per-context YAML contract.

License

Part of the `vasic-digital` toolkit. Distributed under the same terms as the surrounding `vasic-digital` / HelixConstitution submodule family.

Design provenance

The authoritative Phase-0 design (DESIGN / RESEARCH / PLAN) lives in the consuming project's research tree at `docs/research/docs_chain/` and is mirrored into the consuming project, not into this standalone repo. The `docs/` pages here are the self-contained, comprehensive specification of the DESIGNED system.